This traditional model has its benefits, but it's an uphill battle to work against the grain. It's difficult to colocate web and background workers for cost optimizations. And it's near impossible to create in-memory data structures that are shared across the entire application. Effectively, you end up writing applications a certain way, even if it's not how you want to build the application.

The properties of BEAM processes allow for a completely different style of application development. The entry points to the application are the same, but we are no longer limited to a single stack. In fact, we can build applications that consist of many smaller subsystems that have an API used by other parts of our application. Each subsystem has its own stack, garbage collection, and error handling. But they're all deployed as part of the larger application, so complexity is kept to a minimum.

Think of the traditional stack-oriented programming model as a skyscraper. There's a set of doors at the bottom, and you can go all the way to the top before going back out through the doors. But the BEAM programming model lets us build cities instead of skyscrapers. We have the option of building small purpose-driven subsystems that exist for a single purpose, or we can build large monolithic subsystems that operate just like a stack-oriented system does.

In an Elixir app, you'll have many small subsystems that you don't even think about. Talking to the database is a subsystem. Responding to web requests is a subsystem. Background jobs are a subsystem. They all live under the same application and deployment, but they have their own data and performance constraints. These subsystems communicate with each other via message-passing, so they're connected where it matters.

With Elixir, you can truly build a city of subsystems that all work together to create a cohesive application. You can optimize certain parts of the city that become performance bottlenecks. You can add new subsystems without interfering with the rest. It's a beautiful thing when everything comes together in this way, and the BEAM's process model is what makes it possible.

Wrapping Up

Concurrency and parallelism are tough topics at first, but they can be distilled down to a difference between coordinating multiple tasks and executing multiple tasks at the same time. Ruby is a concurrent language, but the Global Virtual Machine Lock prevents it from executing in parallel. The BEAM